

CircleQueue.c 코드 설명

🔖 개요

이 코드는 **원형 큐(Circular Queue)**의 대표적인 구현 예입니다. 배열 기반으로 작성되어 있으며, **삽입/삭제가 순환되도록 설계**되어 공간을 효율적으로 사용할 수 있습니다.

☑️ 주요 구조 및 기능 설명

▶ 구조체 정의

```
#define MAX_QUEUE_SIZE 5

typedef int element;
typedef struct {
    element data[MAX_QUEUE_SIZE];
    int front, rear;
} QueueType;
```

- **front**: 삭제 위치를 가리킴
- **rear**: 삽입 위치를 가리킴
- **data[]**: 원형 큐 저장 배열

▶ 초기화 및 상태 함수

```
void init_queue(QueueType *q);
int is_empty(QueueType *q);
int is_full(QueueType *q);
```

- **front = rear = 0** 으로 초기화
- **비어있음**: `front == rear`
- **가득 참**: `(rear + 1) % MAX == front`

▶ 출력 함수

```
void queue_print(QueueType *q);
```

- **front → rear** 까지 순환하며 출력
- 빈 큐일 경우 아무것도 출력하지 않음

```
QUEUE(front=2 rear=4) = 30 | 40 |
```

▶ 삽입 및 삭제 함수

```
void enqueue(QueueType *q, element item);
element dequeue(QueueType *q);
```

- enqueue: rear를 한 칸 이동 후 저장
- dequeue: front를 한 칸 이동 후 반환
- 모두 % MAX_QUEUE_SIZE로 원형 유지

학습 예시: 작동 순서

```
QueueType q;
init_queue(&q);

enqueue(&q, 10); // rear = 1
enqueue(&q, 20); // rear = 2
enqueue(&q, 30); // rear = 3
queue_print(&q);

dequeue(&q);      // front = 1
enqueue(&q, 40); // rear = 4
queue_print(&q);
```

내부 상태 도식

```
Index:    0   1   2   3   4
Data:     [ ] [10][20][30][40]
Front:           → 1
Rear:                           → 4
```

⚠ 경고 & 참고

- MAX_QUEUE_SIZE 중 실제 사용은 최대 MAX - 1
- 포인터가 뒤섞일 경우 디버깅 어려움 → 정확한 % 연산 필수

평가

- 공간 낭비 없이 큐 사용 가능

- 실제 운영체제나 장비 드라이버 등에서 사용되는 기초 기술
-

파일 명

CircleQueue.c: 배열 기반 원형 큐의 대표적인 구현으로, 큐 관련 개념 학습에 적합한 실습 코드